Guided exercises on assembly with different types of sequencing data

Context

This morning, beverage control arrived with a dramatic `geneX` alert and the alignment team did what alignment teams do best: they compared sequences until the panic became a table. Good news: the exact high-risk synthetic `geneX` allele was not detected in the sequenced beverage isolate.

Bad news: "we did not find that exact allele" is not the same as "we understand what is in this drink". Beverage control is happier, but not yet happy enough to let everyone celebrate with full confidence. The sample still deserves a broader look, and the institute still has way too many unused command-line tools.

In this afternoon practical, you will assemble the same isolate from Illumina, PacBio HiFi, and ultra-long Oxford Nanopore reads, compare the assemblies, and see what each sequencing strategy can and cannot tell you before tonight's tasting committee starts asking nervous questions near the fridge.

Disclaimer and Overview

All reads, genomes, sequences, names, and beverage drama are STILL synthetic. The dataset was designed for a fast and controlled teaching practical. Any resemblance to real beverages, places, institutions, organizers, organisms, or public-health decisions is accidental.

The practical follows a simple assembly workflow: first examine the read datasets, then choose one or more assemblers, evaluate the resulting contigs, and repeat with different data combinations or parameters. The point is not to follow one royal road to the answer. The point is to try things, compare assemblies, and understand why different sequencing technologies give different results.

You may start with one dataset and one assembler, then improve or challenge that result. Try short reads alone, long reads alone, HiFi reads alone, hybrid strategies, different k-mer choices, paired versus unpaired interpretations, or different evaluation tools. The practical is intentionally exploratory: a failed-looking assembly can still teach you something useful if you understand why it failed.

As this morning, you have access to a cheatsheet. Treat it like emergency biscuits: very useful when you are stuck, but a terrible idea as your whole lunch. Weird command choices, disappointing contigs, and suspiciously beautiful results are part of the lesson.

`Reference.fasta` is provided only for reference-based evaluation. Do not open it, inspect it, or use it to guide assembly choices. When the reference-based evaluation step arrives, use it as input to QUAST or similar evaluation tools, not as a sequence to manually study. Looking at the answer key early spoils the fun, and it is also not what happens in practice: most real assembly projects do not come with the exact ground truth waiting in the folder.

Remote Setup

The practical is run on the course VM. Connect via SSH, then activate the environment with conda:

```
conda activate pangenomics
```

The shared input files are in:

`/disk/shared/Assembly_Practical/data`

The optional real-data extension files are in:

`/disk/shared/Assembly_Practical/real_data`

Do not install tools on the VM; all expected tools should already be available in the `pangenomics` environment.

Goal

Your goal is to produce and compare genome assemblies from the same synthetic beverage isolate used in the alignment practical. You should be able to explain why short reads, HiFi reads, and ultra-long ONT reads give different assemblies, and how the evidence from each assembly supports or limits your interpretation.

Data

- Interleaved paired Illumina reads: `/disk/shared/Assembly_Practical/data/Illumina.fastq`

- PacBio HiFi reads: `/disk/shared/Assembly_Practical/data/HiFi.fastq`
- Ultra-long ONT reads: `/disk/shared/Assembly_Practical/data/ONT.fastq`
- Reference genome for evaluation: `/disk/shared/Assembly_Practical/data/Reference.fasta`

Tool Choice

At each step, choose one tool among the proposed options. Groups do not need to use the same tools. The important part is to justify the choice and interpret the output.

Step	Tool options	Expected result
Read inspection	seqkit	Read counts, bases, N50, coverage estimate
Short-read assembly	SPAdes, MEGAHIT, IDBA-UD	Fragmented but accurate draft assembly
ONT assembly	Flye, Raven, Shasta	More contiguous long-read assembly
HiFi assembly	Flye HiFi, HiFiasm	Accurate long-read assembly
Evaluation	QUAST, Meryl, Merqury, Bandage	Contig statistics, k-mer support, graph checks

Step 1 - Inspect the Reads

The synthetic read files are already uncompressed. Use the shared file paths listed in the Data section and compute read statistics.

Tool notes:

- seqkit is the main utility for FASTA/FASTQ inspection in this practical. It can report read counts, total bases, length summaries, N50, and can also convert, filter, sample, sort, and reformat sequence files.

Questions:

- Which dataset has the longest reads?
- Which dataset has the highest coverage?
- What are the number of reads, total bases, read length range, mean read length, and N50 for each dataset?
- What is the approximate coverage of each dataset against a `500 kb` genome?
- Which dataset is most likely to span repeats?

Step 2 - Assemble Short Reads

Choose one short-read assembler: SPAdes, MEGAHIT, or IDBA-UD. Use the interleaved paired Illumina file.

Tool notes:

- SPAdes is a widely used short-read assembler based on de Bruijn graphs. It is a good first choice for small bacterial datasets and has modes for paired reads and hybrid assemblies.
- MEGAHIT is a fast and memory-efficient de Bruijn graph assembler. It is useful when you want a quick comparison with another k-mer graph implementation.
- IDBA-UD is another iterative de Bruijn graph assembler. On this VM it needs FASTA input and the workaround described below.

Questions:

- How many contigs are produced?
- What is the total assembly length?
- What is the N50?
- Does the assembly look complete enough for gene-level analysis?
- Why can short-read assemblies remain fragmented even at high coverage when the genome contains regions that reads cannot place unambiguously?

On this VM, IDBA and IDBA-UD are installed, but they need FASTA input and their paired-link/scaffolding stage is unstable on this dataset. If you choose `idba` or `idba_ud`, use the workaround `--min_pairs 100000000` in addition to your k-mer settings. This effectively disables the problematic paired-link step. Record it as a VM/tool workaround, not as a failure of the read data.

Step 3 - Assemble ONT Long Reads

Choose one ONT assembler: Flye, Raven, or Shasta. Use the ultra-long ONT reads.

Tool notes:

- Flye is a long-read assembler built around repeat graphs. It is a strong default for ONT reads and is useful when you want to see how long reads help with repeated regions.
- Raven is a fast overlap-layout-consensus assembler for long reads. It gives an independent long-read assembly that can be useful even when the result is not the most complete one.
- Shasta is a long-read assembler designed for speed on nanopore-style data. It is useful here as another view of how a sparse and noisy long-read dataset changes the assembly.

Questions:

- Does the long-read assembly have fewer contigs than the short-read assembly?
- Is the total length similar to the short-read assembly?
- Does one very long contig appear?
- What risks come from lower coverage or a higher raw-read error rate?

Step 4 - Assemble HiFi Reads

Choose one HiFi assembler: Flye in HiFi mode or HiFiasm. Use the PacBio HiFi reads.

Tool notes:

- Flye can also assemble accurate long reads using its HiFi mode. This lets you compare the same assembler family on noisy ONT reads and accurate PacBio HiFi reads.
- HiFiasm is an assembler designed for accurate long reads, especially PacBio HiFi. It often produces highly accurate contigs, but a neat-looking assembly still needs evaluation.

Questions:

- How does the HiFi assembly compare with the ONT assembly in contiguity?
- How does it compare with the short-read assembly in fragmentation?
- Which assembly would you use for SNP-level confidence?
- Which assembly would you use to resolve a large repeat or structural question?

Step 5 - Try a Hybrid Assembly

Try one or several hybrid assembly strategies. The point is to test whether combining data types actually improves the result.

Possible strategies include:

- SPAdes with Illumina reads plus ONT long-read support;
- SPAdes with Illumina reads plus HiFi long-read support;
- SPAdes with Illumina reads plus both long-read datasets;
- HiFiasm with HiFi reads plus ultra-long ONT reads;
- Verkko with HiFi and ultra-long ONT reads.

Tool notes:

- SPAdes is the short-read-centered hybrid route: Illumina reads build the graph, and long reads can provide extra linking information.
- HiFiasm and Verkko are long-read-centered hybrid routes: HiFi reads provide accurate long-read sequence, while ultra-long ONT reads can add repeat-spanning information.

Questions:

- Which hybrid strategy did you try, and why?
- Does the hybrid assembly improve contiguity compared with single-technology assemblies?
- Does it agree with the total length suggested by other long-read assemblies?
- When does combining accurate reads and very long reads help resolve a genome?
- Can adding another dataset make the result worse or harder to interpret?
- What would you check before trusting a one-contig result?

Step 6 - Evaluate Assemblies

Once you obtain one or several assemblies, assess their quality. Start without the reference, because in real life you often do not have the exact answer key. Then, only after that, use the provided reference for a second evaluation.

De Novo Evaluation

This evaluation does not use the reference genome.

Available tools:

- seqkit `stats`, to get basic FASTA statistics such as number of sequences, total length, minimum/mean/maximum length, and N50;
- QUAST without `-r`, to summarize contiguity metrics without aligning to a reference;
- Meryl and Merqury, to compare k-mers from the reads with k-mers found in your contigs;
- Bandage, if the assembler produced a GFA graph, to inspect whether the graph is linear, fragmented, or full of unresolved branches.

seqkit and QUAST summarize assemblies from different angles: one reports direct FASTA statistics, the other adds assembly-focused metrics and comparison tables. Meryl and Merqury evaluate k-mer support from the reads, which is useful when there is no reference. Bandage is for looking at assembly graphs when an assembler gives you one, because a graph can explain why contigs stop where they do.

For Merqury, first count k-mers with Meryl from a read set, preferably Illumina or HiFi reads. Then run Merqury with the k-mer database and your assembly. Do not use noisy ONT reads for this unless you are explicitly testing why that is a bad idea.

Questions:

- How many contigs does the assembly contain?
- What are the total length, largest contig, and N50?
- Are frequent read k-mers represented in the assembly?
- Are there many assembly-only k-mers that suggest errors?
- If a graph is available, does it look linear, fragmented, or branched?

Reference-Based Evaluation

This evaluation uses `Reference.fasta`, so do it after the de novo checks. QUAST can align your contigs to the reference and report additional metrics.

Available tool:

- QUAST with `-r Reference.fasta`, to estimate genome fraction, misassemblies, NGA-style metrics, mismatches, and indels.

QUAST can also receive several assemblies at the same time, which is useful when you want to compare the results of different tools or read combinations side by side.

Questions:

- What percentage of the reference is covered by the assembly?
- Are there large or small misassemblies?
- How close are the contigs to the reference in mismatches and indels?
- Does the reference-based evaluation agree with the de novo evaluation?
- Did the reference change your interpretation, or only confirm what you already suspected?

Real-World Assembly (Takes Time)

If you finish the synthetic dataset early, you can repeat part of the workflow on real public reads from a small bacterial genome. The files are already downloaded on the VM.

Shared raw files:

- Illumina MiSeq paired reads: `/disk/shared/Assembly_Practical/real_data/SRR9043691_1.fastq.gz`, `/disk/shared/Assembly_Practical/real_data/SRR9043691_2.fastq.gz`
- Oxford Nanopore reads: `/disk/shared/Assembly_Practical/real_data/SRR10342325_1.fastq.gz`

These files are compressed raw data. Work only inside your own directory and make smaller subsets before assembling. The full datasets are high coverage and will take longer than the synthetic exercise. A target around `50x` coverage is enough for this extension.

For the ONT reads, keeping the longest reads is often more useful than random subsampling; seqkit `sort` can help. If you independently subsample the two Illumina mate files, do not use the subsets in paired-end mode. Treat them as single-end data unless you use a paired-aware subsampling method.

Repeat the same logic as above: inspect the reads, choose one or more assemblers, evaluate the contigs, and compare the result with what happened on the synthetic genome. Real data is less tidy, so expect more decisions about filtering, coverage, runtime, and interpretation.

Final Words

Genome assembly is an inference from incomplete reads. The result depends on read length, coverage, error profile, repeats, assembler assumptions, and the question being asked. Short reads can give strong depth and useful consensus support, but they break when repeats are longer than the information carried by the reads. Long reads can connect distant regions, but coverage and raw-read errors still matter. HiFi reads often give a strong balance between length and accuracy, and hybrid strategies can help when each technology supplies a missing piece.

A careful conclusion should not rely on one metric. Contig count, total length, N50, read support, k-mer consistency, graph structure, and reference-based metrics each describe only part of the result. A one-contig assembly is not automatically correct, and a fragmented assembly is not automatically useless. Always ask what the data could resolve, what the assembler may have simplified, and what evidence would be needed before using the assembly for downstream interpretation.

For the bravest: there is a hidden message in the repeated sequences.

Optional Treasure Hunt

There is a hidden message in the repeated sequences of the synthetic genome. If your assembly preserves those repeats, inspect them.

If you get stuck, ask a teaching assistant for a clue.